

C#.NET



Multithreading

Part-5

Mr. Bangar Raju

 **NARESHO**
technologies

Understanding Thread Locking in C# .NET: A Comprehensive Guide

By Albert Harmon

Published April 12, 2017

🕒 3 min read

[Technology](#)



TLDR This blog post explores thread locking in C# .NET, explaining its importance in multi-threaded environments, demonstrating how to implement it, and discussing its benefits in

preventing unexpected results when multiple threads access shared resources.

In the world of programming, particularly in C#, multi-threading is a powerful feature that allows concurrent execution of code. However, it also introduces challenges, especially when multiple threads attempt to access shared resources simultaneously. This blog post delves into the concept of thread locking, its necessity, and how to implement it effectively in C# .NET.

Recap of Previous Concepts

Before we dive into thread locking, let's briefly recap what we covered in the previous video about the join method. The join method allows one thread to wait for another thread to complete its execution. This is crucial in scenarios where the order of execution matters.

What is Thread Locking?

Thread locking is a mechanism that prevents multiple threads from accessing a shared resource at the same time. This is essential in multi-threaded environments, such as database servers and web servers, where concurrent access can lead to inconsistent or unexpected results.

The Problem with Multi-threading

When multiple threads attempt to access the same resource, context switching occurs. This means that the operating system shares time between threads, which can lead to one thread accessing a resource before another thread has finished using it. This can result in unpredictable behavior, especially when threads are modifying shared data.

Implementing Thread Locking in C#

To illustrate thread locking, let's consider a simple example. We will create a non-static method that prints a message and simulates a delay using `Thread.Sleep``. Here's how we can implement it:

```
public class ThreadLocking
{
    public void Display()
    {
        Console.WriteLine("C# is an object-oriented language.");
        Thread.Sleep(5000);
        Console.WriteLine("Object-oriented language.");
    }
}
```



Calling the Method with Multiple Threads

To demonstrate the issue with multiple threads accessing the same method, we will create three threads that call the `Display` method:

```
class Program
{
    static void Main(string[] args)
    {
        ThreadLocking obj = new ThreadLocking();
        Thread t1 = new Thread(obj.Display);
        Thread t2 = new Thread(obj.Display);
        Thread t3 = new Thread(obj.Display);

        t1.Start();
        t2.Start();
        t3.Start();
    }
}
```

When we run this code, we may see interleaved outputs from the threads, indicating that they are accessing the method simultaneously. This is where thread locking becomes crucial.

Implementing Locking Mechanism

To prevent multiple threads from executing the critical section of code at the same time, we can use the `lock` statement in C#. Here's how to implement it:

```
public class ThreadLocking
{
    private readonly object lockObject = new object();

    public void Display()
    {
        lock (lockObject)
        {
            Console.WriteLine("C# is an object-oriented language");
            Thread.Sleep(5000);
            Console.WriteLine("Object-oriented language.");
        }
    }
}
```

How Locking Works

In this implementation, the `lock` statement ensures that only one thread can enter the critical section of code at a time. If a thread is executing the code inside the lock, other threads attempting to enter will be blocked until the first thread exits the lock. This prevents the unpredictable behavior we observed earlier.

Benefits of Thread Locking

1. **Consistency:** Ensures that shared resources are accessed in a controlled manner, preventing data corruption.
2. **Predictability:** Reduces the chances of unexpected results due to concurrent modifications by multiple threads.
3. **Simplicity:** The `lock` statement provides a straightforward way to manage access to shared resources without complex synchronization mechanisms.

Conclusion

Thread locking is an essential concept in multi-threaded programming in C#. By implementing locking mechanisms, developers can ensure that their applications behave predictably and consistently, even in complex multi-threaded environments. Understanding and applying thread locking will help you avoid common pitfalls associated with concurrent programming.

Thank you for reading! For more insights and tutorials on C#, subscribe to our channel for future updates.